

CoachConnect

Architecture & Product Specification — Brief Edition

Purpose

CoachConnect is a zero-cost sports-coach marketplace for Pakistan. It helps people discover approved coaches, understand services, and request one-to-one online or in-person sessions without the platform collecting payments.

Core product decisions

- The complete approved catalog remains visible; search and recommendations only help rank or narrow it.
- One account can act as an athlete and, after approval, as a coach. Administrator access is granted separately.
- The database—not the browser—enforces privacy, permissions, moderation, and booking conflicts.
- The MVP uses free tiers and local algorithms; no paid AI API, payment processor, Redis, or microservices are required.

Current status

Capability	Status	Boundary
Authentication, catalog, profiles	Live	Supabase-backed public and member experiences.
Natural-language search & recommendations	Live in code	Local deterministic interpretation and explainable ranking.
Availability & booking	Local release candidate	Explicit slots and booking lifecycle are implemented; hosted migration/live verification remain.
Verified ratings & reviews	Planned	Must be tied to completed bookings.
Public deployment	Older release live	Local uncommitted work is not yet represented in production.

Status warning: **built locally does not mean deployed**. The public Vercel site may be behind the current source tree.

1. Product specification

Who uses CoachConnect and what the MVP must do.

Launch baseline

Capability	Status	Boundary
Market	Live baseline	Pakistan: Karachi, Lahore, Islamabad and Rawalpindi.
Currency	Live baseline	Pakistani rupees, shown as Rs 3,000.
Sessions	Live baseline	One-to-one, online or in person.
Payments	Excluded	CoachConnect records prices and policy outcomes but does not charge or refund.
Under-18 use	Blocked for launch	Requires a guardian-managed safeguarding workflow first.

Actors

- **Visitor:** browse, filter, sort, use natural-language search, and view public coach profiles.
- **Member/athlete:** request another coach's slot, manage bookings, and later review a completed session.
- **Approved coach:** maintain a profile, publish slots, respond to requests, add private meeting details, cancel, and complete sessions.
- **Administrator:** approve, request changes, suspend, or restore coach capability; self-review is forbidden.

Primary journeys

Discover	Browse all approved coaches → interpret search → edit filters → compare profiles.
Book	Choose open slot → sign in → request → coach accepts/declines → both schedules agree.
Become a coach	Use existing account → complete application → administrator approval → publish and maintain profile.
Review	Session ends → booking completed → athlete submits one verified review. Planned.

Out of scope

- Online payments, payouts, subscriptions, native apps, chat/video/SMS/WhatsApp, exact public home addresses, group sessions, medical advice, and paid AI services.

2. System architecture

A modular monolith: one web application and one managed backend.

People visitor · member · coach · admin	Next.js UI pages · forms · catalog · schedules	Next.js API validation · sessions · origin checks	Supabase Auth · PostgreSQL · RLS · Storage
---	--	---	--

User request → UI → API → protected database operation → safe response

Frontend

- **Next.js 16 + React 19:** server-rendered pages, direct profile URLs, interactive forms, and API routes in one codebase.
- **TypeScript:** safer booking states, coach lifecycle states, API payloads, and capability combinations.
- **Purpose-built CSS:** distinct visual identity with direct control of responsive and accessible behavior.
- **Mapbox:** optional approximate-area map; the coach list still works if the map fails.

Backend-for-Frontend

- Next.js route handlers validate IDs and input lengths, reject unsafe cross-origin mutations, verify Supabase sessions, and call narrow database functions.
- There is no separate Express/FastAPI service because it would add deployment and authentication complexity without a current scaling need.

Why a modular monolith

The product has one focused domain and a small team. A single deployable application is easier to understand, test, secure, and host. Microservices should be considered only after measured load or independent team ownership justifies them.

Deployment paths

Local development Next.js dev server	Local release hardened Docker	Production Vercel HTTPS/CDN	External backend Supabase Free
--	---	---------------------------------------	--

3. Supabase, data, and security

Why Supabase is the authoritative marketplace backend.

Why Supabase

Capability	Status	Boundary
Auth	Chosen	Avoids building password storage, recovery, and session rotation.
PostgreSQL	Chosen	Relational constraints and transactions fit booking and moderation.
Row-Level Security	Chosen	Owner, participant, public, coach, and admin rules are enforced at the data boundary.
SQL functions / RPC	Chosen	Sensitive lifecycle changes are atomic and narrowly exposed.
Storage	Chosen	Validated coach images use controlled paths and signed URLs.
Free plan	Chosen	Meets the Rs 0 requirement; inactivity and quota limits require clear outage states.

Main data records

- **profiles:** private member record linked to Supabase Auth.
- **coach_applications:** coach profile plus approval/suspension lifecycle.
- **coach_moderation_events:** administrator audit history.
- **coach_availability_slots:** explicit future inventory owned by an approved coach.
- **coach_bookings:** participants, slot, price snapshot, lifecycle, cancellation, and private meeting details.
- **curated_demo_coaches:** clearly labeled illustrative profiles. Reviews remain planned.

Privacy boundary

- Public APIs expose only approved publication-safe fields: name, sports, broad city/area, service details, price, public availability, and later verified reviews.
- Login email, phone, exact home address, precise residential coordinates, private credential files, moderation notes, and participant-only meeting details remain private.
- Unavailable coach profiles use the same not-found response whether missing, draft, rejected, or suspended.

Why not Firebase or custom infrastructure?

PostgreSQL directly supports foreign keys, partial unique indexes, row locks, range-overlap checks, and transactions. A custom database/auth server would require more security and operations work; Firebase would make the relational booking rules less direct.

4. AI and intelligent discovery

Two meaningful zero-cost systems, with honest limitations.

Natural-language search

Example: *Beginner cricket coach in Lahore under Rs 5,000 on Saturday.*

Normalize	Interpret	User control	Search
case · punctuation · currency	aliases · typos · budget · day	editable filters · conflicts	approved coaches only

The parser recognizes supported sports, cities, levels, formats, budgets, days, and specialty terms. It uses aliases and length-aware typo matching, preserves unknown words as normal keywords, and shows conflicts instead of silently inventing a choice.

Explainable recommendations

- A deterministic content-based score considers sport, goal/specialty, level, city or mode, budget, availability, and keyword overlap.
- Results use labels such as Strong match or Good match and show reasons derived only from factors that were actually scored.
- The full approved catalog remains reachable; recommendation ranking never becomes an authorization filter.

Why this approach

Capability	Status	Boundary
Cost	Rs 0	No per-token or API subscription fee.
Privacy	Local	Search text is not sent to a paid model provider.
Trust	Predictable	Cannot invent coaches, credentials, ratings, prices, or slots.
UX	Fast and editable	No model network delay; users can correct interpreted filters.
Testing	Deterministic	Queries, extracted fields, rankings, and reasons can be asserted.

Important limitation

The current implementation is deterministic NLP and recommendation logic, not a trained machine-learning model or hosted LLM. This is intentionally documented. A semantic model should be added only if a representative evaluation set proves that it materially improves search. An evaluator who explicitly requires a learned model may consider the current design a rubric risk.

5. Availability and booking

Correctness is owned by PostgreSQL, not by the calendar display.

Coach publishes explicit slot	Athlete requests open slot	Database locks and rechecks	Both schedules show one agreed state
---	--------------------------------------	---------------------------------------	--

Lifecycle

REQUESTED → **CONFIRMED** or **DECLINED**

REQUESTED / CONFIRMED → **CANCELLED_BY_ATHLETE** or **CANCELLED_BY_COACH**

CONFIRMED → **COMPLETED** after the session end.

Conflict protection

- A partial unique index permits only one active requested/confirmed booking per slot.
- Database functions lock and recheck the coach, slot, approval state, self-booking rule, and schedule overlaps in one transaction.
- Coach slot overlap and athlete overlap with other sessions are rejected.
- Stale or repeated actions return a truthful conflict instead of creating duplicate state.
- Private meeting instructions are set by the coach after confirmation and returned only to participants.

Payment and cancellation

Capability	Status	Boundary
Payment status	NOT_COLLECTED	CoachConnect never holds money.
Athlete cancels ≥24h	FULL_REFUND_DUE	Only if prepaid directly; coach handles it.
Athlete cancels <24h	Outside full-refund window	Policy outcome only; no platform refund.
Coach cancels	FULL_REFUND_DUE	Only if prepaid directly; coach handles it.

Current availability limit

The local release candidate manages explicit slots. Recurring weekly availability rules and time-off exceptions remain planned.

6. Reliability, testing, and next work

What must be true before the local release candidate becomes a verified production release.

Reliability and deployment

- `/api/health` checks process liveness; `/api/ready` checks required Supabase catalog access.
- Docker binds to 127.0.0.1, runs as a non-root user, uses a read-only root filesystem, drops Linux capabilities, and enables no-new-privileges.
- Vercel hosts the Next.js application, but a successful web deployment does not prove that Supabase migrations or real booking flows work.
- Before catalog growth, move filtering/sorting/pagination server-side and add measured database indexes before introducing caches.

Current verification snapshot

Capability	Status	Boundary
ESLint	Passed	Source/framework rules passed.
TypeScript	Passed	Static type checking passed.
Automated tests	198 / 204 passed	One stale demo-copy assertion; five retryable Supabase Auth HTTP 500 failures during disposable-user creation.
Architecture PDF	Verified	Page count, extracted text, and rendered pages checked.

Remaining release work

- Apply and verify booking/availability migrations in the hosted environment, then run production end-to-end tests.
- Implement verified ratings and reviews tied to completed bookings.
- Add recurring weekly availability, time-off exceptions, and Earliest availability sorting.
- Add automated multi-user browser E2E coverage and complete mobile/keyboard checks.
- Move catalog queries and pagination server-side before inventory growth.
- Reconcile older project-report and phase wording with current demo and feature status.

Source-of-truth files

- `PRODUCT_REQUIREMENTS.md` — required behavior; `PLAN.md` — delivery gates; `SCOPE.md` — accepted changes.
- `README.md` — current setup/status; `ARCHITECTURE_AND_PRODUCT_SPECIFICATION.md` — detailed technical source.
- `supabase/migrations/` — database/security rules; `src/app/api/` — server boundaries; `tests/` — verification evidence.

No GitHub push, production deployment, hosted migration, or paid service change is included in this document task.